

CO4 AT2 – Industry Problem Solving

DSA 0201 - Computer vision with Open cv

Name: K. Madhulika

Reg no: 192524190

1: Automated Defect Detection in Manufacturing

A precision engineering company wants to automatically detect defects in mechanical components using computer vision. Parts vary slightly in shape due to manufacturing tolerances.

Questions:

- 1. How can PCA-based shape priors and contour-based representations be used to model standard component shapes?**
- 2. Propose a pipeline using machine learning (SVM, GMM) or deep learning (CNN) to classify defective vs. non-defective parts.**
- 3. Discuss methods to handle noise, partial occlusions, and varying illumination in industrial images.**

1. Use of PCA-based Shape Priors and Contour Representations

In manufacturing, components may have small variations because of manufacturing tolerances. PCA (Principal Component Analysis) can be used to learn the normal shape variation of defect-free components.

Process:

1. Capture images of several defect-free components.
2. Segment the component from the background.
3. Extract its contour/boundary using techniques such as Canny edge detection.
4. Convert the contours into fixed-size feature vectors.

5. Apply PCA to these feature vectors.
6. PCA identifies the main patterns of shape variation in normal components.
7. A new component is compared with the PCA shape model.
8. If its shape differs significantly from the learned normal range, it can be considered potentially defective.

Contour-based representation describes the outer boundary of a component using features such as:

- Contour points
- Perimeter
- Area
- Circularity
- Shape moments
- Curvature

Thus, PCA provides a compact model of normal shapes, while contour features help identify abnormalities such as cracks, missing parts, dents, or deformation.

2. Proposed Machine Learning / Deep Learning Pipeline

A suitable defect-detection pipeline can be:

Image Acquisition → Preprocessing → Segmentation → Feature Extraction → Classification
→ Defect Decision

Step 1: Image Acquisition

Capture images of mechanical components using an industrial camera under controlled lighting.

Step 2: Preprocessing

Improve image quality using:

- Gaussian filtering for noise reduction
- Histogram equalization for illumination correction

- Image resizing and normalization

Step 3: Segmentation

Separate the component from the background using thresholding, edge detection, or segmentation algorithms.

Step 4: Feature Extraction

Extract:

- PCA shape features
- Contour features
- Texture features
- Edge information

Step 5: Classification

SVM:

An SVM can classify the extracted features into Defective and Non-defective classes. It works well when the dataset is relatively small and the extracted features are meaningful.

GMM:

A Gaussian Mixture Model can model the distribution of normal components. A new component with a very low probability under the normal model can be marked as an anomaly.

CNN:

For a large image dataset, a Convolutional Neural Network can learn features directly from images and classify them as defective or non-defective. CNNs are particularly useful for complex defects such as cracks and surface damage.

A practical approach is:

Camera → Preprocessing → CNN/SVM → Defect Classification → Quality Control

The output can be:

- Class 1: Non-defective
- Class 2: Defective

3. Handling Noise, Partial Occlusions, and Varying Illumination

A. Noise

Industrial images may contain sensor noise, dust, or unwanted small regions.

Methods:

- Gaussian or median filtering
- Morphological opening and closing
- Noise removal using connected-component analysis
- Image denoising before feature extraction

B. Partial Occlusion

Sometimes part of the component may be hidden by another object or fixture.

Methods:

- Use robust contour descriptors that do not depend on the complete contour.
- Use local features instead of relying only on the complete shape.
- Train CNN models using partially occluded training images.
- Use data augmentation such as random cropping and masking.

C. Varying Illumination

Different lighting conditions can change the brightness and appearance of the component.

Methods:

- Use controlled industrial lighting.
- Apply histogram equalization or CLAHE.
- Normalize image intensity.
- Convert RGB images to grayscale or suitable color spaces.
- Train CNN models with images captured under different lighting conditions.

Conclusion

PCA and contour representations can create a reliable model of normal component shapes, making it possible to detect abnormal shapes caused by manufacturing defects. SVM and GMM are suitable for feature-based classification, while CNNs can automatically learn complex visual features from images. Proper preprocessing, illumination correction, noise removal, and data augmentation make the system more robust in real industrial environments.

2: Autonomous Vehicle Obstacle Recognition

A self-driving car company needs to detect rare or unexpected obstacles on urban roads. The labelled data for these obstacles is limited.

Questions:

- 1. Explain how data augmentation using GANs can help generate synthetic images for training.**
- 2. Compare the effectiveness of CNNs versus Vision Transformers for detecting small or sparse objects in high-resolution images.**
- 3. Suggest a real-time inference pipeline for obstacle detection, considering computational constraints in embedded systems.**

1. Data Augmentation Using GANs

In autonomous vehicles, rare obstacles such as fallen objects, unusual vehicles, animals, or road debris may have very few labelled images. Generative Adversarial Networks (GANs) can generate realistic synthetic images to increase the training dataset.

A GAN consists of two main networks:

- **Generator:** Creates synthetic obstacle images from random input.
- **Discriminator:** Tries to distinguish between real and generated images.

The two networks compete with each other until the generator produces realistic images.

Process:

Real obstacle images → Train GAN → Generate synthetic images → Combine real + synthetic data → Train detection model

For example, if only 100 images of a rare road obstacle are available, GANs can generate additional variations with different:

- Positions
- Sizes
- Viewing angles
- Backgrounds
- Lighting conditions
- Weather conditions

This improves model diversity and can reduce overfitting. However, synthetic images should be validated carefully because unrealistic GAN-generated images can introduce incorrect patterns into the training data.

2. CNN vs Vision Transformer

Feature	CNN	Vision Transformer (ViT)
Basic principle	Uses convolution filters	Uses attention mechanism
Local features	Very good	Good
Small objects	Generally effective	Can be effective with suitable resolution
Global context	Limited compared with Transformers	Excellent
Computational requirement	Usually lower	Usually higher
Training data requirement	Relatively lower	Usually requires more data
Embedded deployment	Easier	More difficult

Feature	CNN	Vision Transformer (ViT)
High-resolution images	Efficient with optimized architectures	Can be computationally expensive

CNNs:

CNNs are good at detecting local patterns such as edges, corners, and textures. They are often more computationally efficient and therefore suitable for real-time autonomous-vehicle systems. Multi-scale CNN-based detectors can detect small objects effectively.

Vision

Transformers:

Vision Transformers use self-attention to understand relationships between different parts of an image. This can help when a small obstacle needs to be interpreted using its surrounding road context. However, high-resolution images can make Transformers computationally expensive.

Conclusion:

For an embedded vehicle with limited computing power, an optimized CNN-based detector is generally a practical choice. A Vision Transformer or hybrid CNN-Transformer model may provide better contextual understanding when sufficient computational resources are available.

3. Real-Time Inference Pipeline

A suitable real-time obstacle detection pipeline is:

Camera/Sensor → Preprocessing → Object Detection → Tracking → Decision → Vehicle Control

Step 1: Sensor Input

Use cameras, and optionally LiDAR/radar, to capture the road environment.

Step 2: Preprocessing

- Resize images to an appropriate resolution.
- Normalize pixel values.
- Remove unnecessary image information.
- Apply image enhancement when required.

Step 3: Object Detection

Use a lightweight real-time object detector such as a compact YOLO-family model or another optimized detector.

The model identifies:

- Cars
- Pedestrians
- Motorcycles
- Animals
- Road debris
- Unexpected obstacles

Step 4: Object Tracking

Track detected objects across consecutive frames using a lightweight tracking algorithm. This avoids repeatedly detecting the same object from scratch.

Step 5: Confidence and Distance Estimation

The system checks detection confidence and estimates the obstacle's position and distance using camera geometry or additional sensors.

Step 6: Decision Making

If an obstacle is detected in the vehicle's path, the system sends information to the vehicle-control system for actions such as:

- Slow down
- Stop
- Change lane
- Avoid the obstacle

Computational Constraints

For embedded systems, the model should be optimized using:

- Model quantization – use lower-precision calculations such as INT8.
- Pruning – remove unnecessary model parameters.
- Knowledge distillation – train a smaller model using a larger model as a teacher.
- Reduced input resolution – process images at a suitable resolution.
- GPU/NPU acceleration – use available hardware accelerators.
- Frame skipping or adaptive processing – avoid processing every frame when unnecessary.

Overall Pipeline

Camera → Image Preprocessing → Lightweight CNN/Detector → Obstacle Detection → Object Tracking → Distance/Confidence Check → Collision Risk → Vehicle Action

Conclusion:

GANs can increase the availability of rare-obstacle training data, while CNNs provide an efficient solution for real-time detection. Vision Transformers offer strong global-context understanding but generally require more computational resources. For embedded autonomous vehicles, a lightweight detector combined with model compression and hardware acceleration provides a practical balance between accuracy, speed, and computational cost.